

SOCKETS: UNIX DOMAIN

The Linux Programming Interface — Michael Kerrisk

Chương này xem xét cách sử dụng UNIX domain sockets, cho phép giao tiếp giữa các process trên cùng một host. Chúng ta thảo luận về việc sử dụng cả stream sockets và datagram sockets trong UNIX domain. Chúng ta cũng mô tả cách dùng file permission để kiểm soát quyền truy cập vào UNIX domain sockets, cách dùng `socketpair()` để tạo một cặp UNIX domain sockets đã được kết nối, và Linux abstract socket namespace.

57.1 Địa chỉ UNIX Domain Socket: `struct sockaddr_un`

Trong UNIX domain, địa chỉ socket có dạng một pathname, và cấu trúc địa chỉ socket domain-specific được định nghĩa như sau:

```
struct sockaddr_un {
    sa_family_t  sun_family;    /* Luôn là AF_UNIX */
    char         sun_path[108]; /* Pathname socket kết thúc bằng null */
};
```

Tiền tố `sun_` trong các trường của cấu trúc `sockaddr_un` không liên quan gì đến Sun Microsystems; mà bắt nguồn từ *socket unix*.

SUSV3 không chỉ định kích thước của trường `sun_path`. Các triển khai BSD đầu tiên dùng 108 và 104 bytes, và một triển khai đương đại (HP-UX 11) dùng 92 bytes. Các ứng dụng portable nên code theo giá trị thấp hơn này, và dùng `snprintf()` hoặc `strncpy()` để tránh buffer overrun khi ghi vào trường này.

Để bind một UNIX domain socket vào một địa chỉ, chúng ta khởi tạo cấu trúc `sockaddr_un`, sau đó truyền con trỏ (đã cast) tới cấu trúc này làm tham số `addr` cho `bind()`, và đặt `addrlen` bằng kích thước của cấu trúc, như trong Listing 57-1.

Listing 57-1: Binding a UNIX domain socket

```
const char *SOCKNAME = "/tmp/mysock";
int sfd;
struct sockaddr_un addr;

sfd = socket(AF_UNIX, SOCK_STREAM, 0); /* Tạo socket */
if (sfd == -1)
    errExit("socket");

memset(&addr, 0, sizeof(struct sockaddr_un)); /* Xóa trắng cấu trúc */
addr.sun_family = AF_UNIX;                    /* Địa chỉ UNIX domain */
strncpy(addr.sun_path, SOCKNAME, sizeof(addr.sun_path) - 1);

if (bind(sfd, (struct sockaddr *) &addr, sizeof(struct sockaddr_un)) == -1)
    errExit("bind");
```

Việc dùng `memset()` trong Listing 57-1 đảm bảo tất cả các trường của cấu trúc có giá trị 0. (Lời gọi `strncpy()` tiếp theo tận dụng điều này bằng cách chỉ định tham số cuối nhỏ hơn một so với

kích thước trường `sun_path`, để đảm bảo trường này luôn có một null byte kết thúc.) Dùng `memset()` để xóa toàn bộ cấu trúc, thay vì khởi tạo từng trường riêng lẻ, còn đảm bảo rằng các trường không chuẩn do một số triển khai cung cấp cũng được khởi tạo về 0.

Hàm `bzero()` có nguồn gốc từ BSD là một lựa chọn thay thế cho `memset()` để xóa nội dung của cấu trúc. SUSv3 chỉ định `bzero()` và `bcopy()` liên quan (tương tự `memmove()`), nhưng đánh dấu cả hai là LEGACY, lưu ý rằng `memset()` và `memmove()` được ưu tiên hơn. SUSv4 đã loại bỏ đặc tả của `bzero()` và `bcopy()`.

Khi được dùng để bind một UNIX domain socket, `bind()` tạo ra một entry trong file system. (Do đó, một thư mục được chỉ định như một phần của socket pathname cần phải có thể truy cập và ghi được.) Ownership của file được xác định theo các quy tắc thông thường của việc tạo file (Mục 15.3.1). File được đánh dấu là socket. Khi `stat()` được áp dụng lên pathname này, nó trả về giá trị `S_IFSOCK` trong thành phần file-type của trường `st_mode` của cấu trúc `stat` (Mục 15.1). Khi liệt kê bằng `ls -l`, một UNIX domain socket được hiển thị với kiểu `s` ở cột đầu tiên, và `ls -F` thêm dấu bằng (=) vào sau socket pathname.

Mặc dù UNIX domain sockets được nhận dạng qua pathname, I/O trên các socket này không liên quan đến các thao tác trên thiết bị lưu trữ bên dưới.

Các điểm đáng lưu ý khi bind một UNIX domain socket:

- Không thể bind một socket vào một pathname đã tồn tại (`bind()` thất bại với lỗi `EADDRINUSE`).
- Thông thường nên bind socket vào một absolute pathname. Nếu dùng relative pathname, ứng dụng muốn `connect()` đến socket đó phải biết thư mục làm việc hiện tại của ứng dụng đã thực hiện `bind()`.
- Một socket chỉ có thể được bind vào một pathname duy nhất; ngược lại, một pathname chỉ có thể được bind vào một socket duy nhất.
- Không thể dùng `open()` để mở một socket.
- Khi socket không còn cần thiết, entry pathname của nó có thể (và thông thường nên) được xóa bằng `unlink()` (hoặc `remove()`).

Trong hầu hết các chương trình ví dụ, chúng ta bind UNIX domain sockets vào các pathname trong thư mục `/tmp`, vì thư mục này thường có mặt và có thể ghi được trên mọi hệ thống. Điều này giúp người đọc dễ dàng chạy các chương trình mà không cần chỉnh sửa socket pathnames. Tuy nhiên, cần lưu ý rằng đây thường không phải là kỹ thuật thiết kế tốt. Như đã chỉ ra trong Mục 38.7, việc tạo file trong các thư mục có thể ghi công khai như `/tmp` có thể dẫn đến nhiều lỗ hổng bảo mật. Ví dụ, bằng cách tạo một pathname trong `/tmp` với cùng tên như socket của ứng dụng, ta có thể thực hiện một cuộc tấn công denial-of-service đơn giản. Các ứng dụng thực tế nên bind UNIX domain sockets vào các absolute pathname trong các thư mục được bảo mật phù hợp.

57.2 Stream Sockets trong UNIX Domain

Chúng ta sẽ trình bày một ứng dụng client-server đơn giản sử dụng stream sockets trong UNIX domain. Chương trình client (Listing 57-4) kết nối đến server, và dùng kết nối đó để chuyển dữ liệu từ standard input của nó đến server. Chương trình server (Listing 57-3) chấp nhận các kết

nối client, và chuyển tất cả dữ liệu được gửi trên kết nối bởi client ra standard output. Server là một ví dụ đơn giản về *iterative server* — một server xử lý từng client một trước khi chuyển sang client tiếp theo. (Chúng ta xem xét thiết kế server chi tiết hơn trong Chương 60.)

Listing 57-2 là header file được dùng bởi cả hai chương trình này.

Listing 57-2: Header file cho us_xfr_sv.c và us_xfr_cl.c [sockets/us_xfr.h]

```
#include <sys/un.h>
#include <sys/socket.h>
#include "tspi_hdr.h"

#define SV_SOCKET_PATH "/tmp/us_xfr"
#define BUF_SIZE      100
```

Trong các trang tiếp theo, chúng ta trình bày source code của server và client, rồi thảo luận chi tiết về các chương trình này và đưa ra ví dụ thực thi.

Listing 57-3: Server UNIX domain stream socket đơn giản [sockets/us_xfr_sv.c]

```
#include "us_xfr.h"

#define BACKLOG 5

int
main(int argc, char *argv[])
{
    struct sockaddr_un addr;
    int sfd, cfd;
    ssize_t numRead;
    char buf[BUF_SIZE];

    sfd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sfd == -1)
        errExit("socket");

    /* Xây dựng địa chỉ socket server, bind socket vào đó,
       và đánh dấu đây là listening socket */
    if (remove(SV_SOCKET_PATH) == -1 && errno != ENOENT)
        errExit("remove-%s", SV_SOCKET_PATH);

    memset(&addr, 0, sizeof(struct sockaddr_un));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SV_SOCKET_PATH, sizeof(addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &addr,
            sizeof(struct sockaddr_un)) == -1)
        errExit("bind");

    if (listen(sfd, BACKLOG) == -1)
        errExit("listen");

    for (;;) { /* Xử lý các kết nối client theo kiểu iterative */

        /* Chấp nhận kết nối. Kết nối được trả về trên socket mới 'cfd';
           listening socket ('sfd') vẫn mở để chấp nhận thêm kết nối. */
```

```

        cfd = accept(sfd, NULL, NULL);
        if (cfd == -1)
            errExit("accept");

        /* Chuyển dữ liệu từ connected socket sang stdout cho đến EOF */
        while ((numRead = read(cfd, buf, BUF_SIZE)) > 0)
            if (write(STDOUT_FILENO, buf, numRead) != numRead)
                fatal("partial/failed write");

        if (numRead == -1)
            errExit("read");

        if (close(cfd) == -1)
            errExit("close");
    }
}

```

Listing 57-4: Client UNIX domain stream socket đơn giản [sockets/us_xfr_cl.c]

```

#include "us_xfr.h"

int
main(int argc, char *argv[])
{
    struct sockaddr_un addr;
    int sfd;
    ssize_t numRead;
    char buf[BUF_SIZE];

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); /* Tạo client socket */
    if (sfd == -1)
        errExit("socket");

    /* Xây dựng địa chỉ server và thực hiện kết nối */
    memset(&addr, 0, sizeof(struct sockaddr_un));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, SV_SOCKET_PATH, sizeof(addr.sun_path) - 1);

    if (connect(sfd, (struct sockaddr *) &addr,
                sizeof(struct sockaddr_un)) == -1)
        errExit("connect");

    /* Sao chép stdin vào socket */
    while ((numRead = read(STDIN_FILENO, buf, BUF_SIZE)) > 0)
        if (write(sfd, buf, numRead) != numRead)
            fatal("partial/failed write");

    if (numRead == -1)
        errExit("read");

    exit(EXIT_SUCCESS); /* Đóng socket; server nhận EOF */
}

```

Chương trình server (Listing 57-3) thực hiện các bước sau:

- Tạo một socket.
- Xóa bất kỳ file nào đang tồn tại với cùng pathname mà chúng ta muốn bind socket vào.
- Khởi tạo cấu trúc `sockaddr_un`, bind socket vào địa chỉ đó, và đánh dấu socket là listening socket.
- Thực thi một vòng lặp vô hạn để xử lý các yêu cầu client đến. Mỗi lần lặp thực hiện:
 - Chấp nhận một kết nối, thu được socket mới `cfd` cho kết nối.
 - Đọc tất cả dữ liệu từ connected socket và ghi ra standard output.
 - Đóng connected socket `cfd`.

Server phải được kết thúc thủ công (ví dụ: bằng cách gửi cho nó một signal).

Chương trình client (Listing 57-4) thực hiện các bước sau:

- Tạo một socket.
- Xây dựng cấu trúc địa chỉ cho socket của server và connect đến socket tại địa chỉ đó.
- Thực thi một vòng lặp sao chép standard input vào kết nối socket. Khi gặp end-of-file ở standard input, client kết thúc, kết quả là socket của nó được đóng và server nhận end-of-file khi đọc từ socket ở đầu kia của kết nối.

Phiên shell sau minh họa việc dùng các chương trình này. Chúng ta bắt đầu bằng cách chạy server ở nền:

```
$ ./us_xfr_sv > b &
[1] 9866
$ ls -lF /tmp/us_xfr          Kiểm tra socket file bằng ls
srwxr-xr-x 1 mtk users 0 Jul 18 10:48 /tmp/us_xfr=
```

Sau đó tạo file test để dùng làm input cho client, và chạy client:

```
$ cat *.c > a
$ ./us_xfr_cl < a             Client lấy input từ file test
```

Lúc này, client đã hoàn thành. Bây giờ chúng ta kết thúc server và kiểm tra output của server khớp với input của client:

```
$ kill %1                     Kết thúc server
[1]+  Terminated  ./us_xfr_sv >b
$ diff a b
$
```

Lệnh `diff` không tạo ra output, cho thấy file input và output hoàn toàn giống nhau.

Lưu ý rằng sau khi server kết thúc, socket pathname vẫn tiếp tục tồn tại. Đó là lý do tại sao server dùng `remove()` để xóa bất kỳ instance nào đang tồn tại của socket pathname trước khi gọi `bind()`. (Giả sử chúng ta có quyền phù hợp, lời gọi `remove()` này sẽ xóa bất kỳ loại file nào với pathname này, ngay cả khi nó không phải là socket.) Nếu không làm vậy, lời gọi `bind()` sẽ thất bại nếu một lần gọi trước đó của server đã tạo socket pathname này.

57.3 Datagram Sockets trong UNIX Domain

Trong mô tả tổng quát về datagram sockets ở Mục 56.6, chúng ta đã nói rằng giao tiếp bằng datagram sockets là không tin cậy. Điều này đúng với các datagrams truyền qua mạng. Tuy nhiên, với UNIX domain sockets, việc truyền

datagram được thực hiện trong kernel và là **đáng tin cậy**. Tất cả các message đều được giao đúng thứ tự và không bị trùng lặp.

Kích thước datagram tối đa cho UNIX domain datagram sockets

SUSv3 không chỉ định kích thước tối đa cho datagrams được gửi qua UNIX domain socket. Trên Linux, chúng ta có thể gửi các datagrams khá lớn. Các giới hạn được kiểm soát thông qua socket option `SO_SNDBUF` và nhiều file `/proc`, như được mô tả trong trang manual `socket(7)`. Tuy nhiên, một số triển khai UNIX khác áp đặt giới hạn thấp hơn, chẳng hạn như 2048 bytes. Các ứng dụng portable dùng UNIX domain datagram sockets nên cân nhắc áp đặt giới hạn trên thấp cho kích thước datagrams được sử dụng.

Chương trình ví dụ

Listing 57-6 và Listing 57-7 trình bày một ứng dụng client-server đơn giản dùng UNIX domain datagram sockets. Cả hai chương trình đều sử dụng header file trong Listing 57-5.

Listing 57-5: Header file cho `ud_ucase_sv.c` và `ud_ucase_cl.c` [`sockets/ud_ucase.h`]

```
#include <sys/un.h>
#include <sys/socket.h>
#include <ctype.h>
#include "tlpi_hdr.h"

#define BUF_SIZE      10      /* Kích thước tối đa message trao đổi
                               giữa client và server */

#define SV_SOCKET_PATH "/tmp/ud_ucase"
```

Chương trình server (Listing 57-6) trước tiên tạo socket và bind vào well-known address. (Trước đó, server unlink pathname khớp với địa chỉ đó, phòng trường hợp pathname đã tồn tại.) Server sau đó vào vòng lặp vô hạn, dùng `recvfrom()` để nhận datagrams từ các client, chuyển đổi văn bản nhận được thành chữ hoa, và trả văn bản đã chuyển đổi lại cho client bằng địa chỉ thu được qua `recvfrom()`.

Chương trình client (Listing 57-7) tạo socket và bind socket vào một địa chỉ, để server có thể gửi reply. Địa chỉ client được làm cho duy nhất bằng cách đưa process ID của client vào trong pathname. Client sau đó lặp qua, gửi mỗi đối số command-line của nó như một message riêng biệt đến server. Sau mỗi lần gửi, client đọc phản hồi từ server và hiển thị lên standard output.

Listing 57-6: Server dùng UNIX domain datagram socket [`sockets/ud_ucase_sv.c`]

```
#include "ud_ucase.h"

int
main(int argc, char *argv[])
{
    struct sockaddr_un svaddr, claddr;
    int sfd, j;
    ssize_t numBytes;
    socklen_t len;
    char buf[BUF_SIZE];

    sfd = socket(AF_UNIX, SOCK_DGRAM, 0); /* Tạo server socket */
    if (sfd == -1)
        errExit("socket");
```

```

/* Xây dựng well-known address và bind server socket vào đó */
if (remove(SV_SOCK_PATH) == -1 && errno != ENOENT)
    errExit("remove-%s", SV_SOCK_PATH);

memset(&svaddr, 0, sizeof(struct sockaddr_un));
svaddr.sun_family = AF_UNIX;
strncpy(svaddr.sun_path, SV_SOCK_PATH, sizeof(svaddr.sun_path) - 1);

if (bind(sfd, (struct sockaddr *) &svaddr,
        sizeof(struct sockaddr_un)) == -1)
    errExit("bind");

/* Nhận messages, chuyển thành chữ hoa, và trả về cho client */
for (;;) {
    len = sizeof(struct sockaddr_un);
    numBytes = recvfrom(sfd, buf, BUF_SIZE, 0,
                        (struct sockaddr *) &claddr, &len);
    if (numBytes == -1)
        errExit("recvfrom");

    printf("Server received %ld bytes from %s\n",
           (long) numBytes, claddr.sun_path);

    for (j = 0; j < numBytes; j++)
        buf[j] = toupper((unsigned char) buf[j]);

    if (sendto(sfd, buf, numBytes, 0,
               (struct sockaddr *) &claddr, len) != numBytes)
        fatal("sendto");
}
}

```

Listing 57-7: Client dùng UNIX domain datagram socket [sockets/ud_ucase_cl.c]

```

#include "ud_ucase.h"

int
main(int argc, char *argv[])
{
    struct sockaddr_un svaddr, claddr;
    int sfd, j;
    size_t msgLen;
    ssize_t numBytes;
    char resp[BUF_SIZE];

    if (argc < 2 || strcmp(argv[1], "--help") == 0)
        usageErr("%s msg...\n", argv[0]);

    /* Tạo client socket; bind vào pathname duy nhất (dựa trên PID) */
    sfd = socket(AF_UNIX, SOCK_DGRAM, 0);
    if (sfd == -1)
        errExit("socket");

    memset(&claddr, 0, sizeof(struct sockaddr_un));

```

```

claddr.sun_family = AF_UNIX;
snprintf(claddr.sun_path, sizeof(claddr.sun_path),
         "/tmp/ud_ucose_cl.%ld", (long) getpid());

if (bind(sfd, (struct sockaddr *) &claddr,
        sizeof(struct sockaddr_un)) == -1)
    errExit("bind");

/* Xây dựng địa chỉ server */
memset(&svaddr, 0, sizeof(struct sockaddr_un));
svaddr.sun_family = AF_UNIX;
strncpy(svaddr.sun_path, SV_SOCKET_PATH, sizeof(svaddr.sun_path) - 1);

/* Gửi messages đến server; hiển thị responses lên stdout */
for (j = 1; j < argc; j++) {
    msgLen = strlen(argv[j]);
    if (sendto(sfd, argv[j], msgLen, 0,
              (struct sockaddr *) &svaddr,
              sizeof(struct sockaddr_un)) != msgLen)
        fatal("sendto");

    numBytes = recvfrom(sfd, resp, BUF_SIZE, 0, NULL, NULL);
    if (numBytes == -1)
        errExit("recvfrom");

    printf("Response %d: %.*s\n", j, (int) numBytes, resp);
}

remove(claddr.sun_path); /* Xóa socket pathname của client */
exit(EXIT_SUCCESS);
}

```

Phiên shell sau minh họa việc dùng hai chương trình này:

```

$ ./ud_ucose_sv &
[1] 20113
$ ./ud_ucose_cl hello world           Gửi 2 message đến server
Server received 5 bytes from /tmp/ud_ucose_cl.20150
Response 1: HELLO
Server received 5 bytes from /tmp/ud_ucose_cl.20150
Response 2: WORLD
$ ./ud_ucose_cl 'long message'       Gửi 1 message dài hơn đến server
Server received 10 bytes from /tmp/ud_ucose_cl.20151
Response 1: LONG MESSA
$ kill %1                             Kết thúc server

```

Lần gọi thứ hai của chương trình client được thiết kế để cho thấy rằng khi lời gọi `recvfrom()` chỉ định một độ dài (`BUF_SIZE`, được định nghĩa trong Listing 57-5 với giá trị 10) ngắn hơn kích thước message, message sẽ bị cắt bớt lặt lể. Chúng ta có thể thấy điều này xảy ra vì server in ra rằng nó chỉ nhận được 10 bytes, trong khi message được client gửi đi gồm 12 bytes.

57.4 Quyền truy cập UNIX Domain Socket (Socket Permissions)

Ownership và permissions của socket file xác định process nào có thể giao tiếp với socket đó:

- Để connect đến một UNIX domain stream socket, cần có quyền **write** trên socket file.
- Để gửi datagram đến một UNIX domain datagram socket, cần có quyền **write** trên socket file.

Ngoài ra, cần có quyền execute (search) trên mỗi thư mục trong socket pathname.

Mặc định, một socket được tạo (bởi `bind()`) với tất cả quyền được cấp cho owner (user), group, và other. Để thay đổi điều này, chúng ta có thể gọi `umask()` trước lời gọi `bind()` để vô hiệu hoá các quyền mà chúng ta không muốn cấp.

Một số hệ thống bỏ qua permissions trên socket file (SUSv3 cho phép điều này). Do đó, chúng ta không thể dùng socket file permissions một cách portable để kiểm soát quyền truy cập vào socket, mặc dù chúng ta có thể dùng permissions trên thư mục chứa socket cho mục đích này theo cách portable.

57.5 Tạo một Cặp Socket Đã Kết nối: `socketpair()`

Đôi khi, sẽ hữu ích khi một process tạo một cặp sockets và kết nối chúng lại với nhau. Điều này có thể thực hiện bằng hai lời gọi `socket()`, một lời gọi `bind()`, và sau đó là các lời gọi `listen()`, `connect()`, và `accept()` (đối với stream sockets), hoặc một lời gọi `connect()` (đối với datagram sockets). System call `socketpair()` cung cấp cách viết tắt cho thao tác này.

```
#include <sys/socket.h>
```

```
int socketpair(int domain, int type, int protocol, int sockfd[2]);
```

Trả về 0 khi thành công, hoặc -1 khi lỗi

System call `socketpair()` chỉ có thể dùng trong UNIX domain; tức là `domain` phải được chỉ định là `AF_UNIX`. (Hạn chế này áp dụng trên hầu hết các triển khai, và có lý vì cặp socket được tạo ra trên một host duy nhất.) Loại socket có thể được chỉ định là `SOCK_DGRAM` hoặc `SOCK_STREAM`. Tham số `protocol` phải được đặt là 0. Mảng `sockfd` trả về các file descriptor tham chiếu đến hai connected sockets.

Chỉ định `type` là `SOCK_STREAM` tạo ra tương đương với một *bidirectional pipe* (còn được gọi là *stream pipe*). Mỗi socket có thể dùng cho cả đọc và ghi, và các kênh dữ liệu riêng biệt chảy theo mỗi hướng giữa hai sockets. (Trên các triển khai BSD-derived, `pipe()` được triển khai dưới dạng lời gọi `socketpair()`.)

Thông thường, một socket pair được dùng theo cách tương tự như một pipe. Sau lời gọi `socketpair()`, process tạo ra một child thông qua `fork()`. Child kế thừa bản sao các file descriptor của parent, bao gồm các descriptor tham chiếu đến socket pair. Do đó, parent và child có thể dùng socket pair cho IPC.

Một điểm khác biệt giữa dùng `socketpair()` so với tạo thủ công một cặp connected sockets là các sockets không được bind vào bất kỳ địa chỉ nào. Điều này có thể giúp tránh một loạt lỗi hỏng bảo mật, vì các sockets không hiển thị với bất kỳ process nào khác.

Từ kernel 2.6.27, Linux cung cấp cách dùng thứ hai cho tham số `type`, bằng cách cho phép OR hai cờ không chuẩn với socket type. Cờ `SOCK_CLOEXEC` khiến kernel bật cờ close-on-exec (`FD_CLOEXEC`) cho cả hai file descriptor mới. Cờ `SOCK_NONBLOCK` khiến kernel đặt cờ `O_NONBLOCK` trên cả hai open file description

bên dưới, giúp tất cả thao tác I/O trên socket sau này là nonblocking, tránh phải gọi thêm `fcntl()` để đạt kết quả tương tự.

57.6 Linux Abstract Socket Namespace

Abstract namespace là tính năng đặc thù của Linux cho phép bind một UNIX domain socket vào một tên mà không cần tên đó được tạo trong file system. Điều này mang lại một số lợi thế tiềm năng:

- Chúng ta không cần lo lắng về xung đột tên với các tên đã tồn tại trong file system.
- Không cần phải unlink socket pathname khi đã dùng xong socket. Abstract name được tự động xóa khi socket được đóng.
- Không cần tạo file-system pathname cho socket. Điều này có thể hữu ích trong môi trường chroot, hoặc khi không có quyền ghi vào file system.

Để tạo một abstract socket binding, ta chỉ định ký tự đầu tiên (`sun_path[0]`) của địa chỉ là một **null byte** (`\0`). Điều này phân biệt abstract socket names với các UNIX domain socket pathname thông thường, vốn gồm một chuỗi một hoặc nhiều byte khác null được kết thúc bằng null byte. Các byte còn lại của trường `sun_path` sau đó định nghĩa abstract name cho socket. Các byte này được diễn giải trọn vẹn, không phải như một null-terminated string.

Listing 57-8 minh họa việc tạo một abstract socket binding.

*Listing 57-8: Tạo một abstract socket binding [from
sockets/us_abstract_bind.c]*

```
struct sockaddr_un addr;

memset(&addr, 0, sizeof(struct sockaddr_un)); /* Xóa trắng cấu trúc địa chỉ */
addr.sun_family = AF_UNIX;                  /* UNIX domain address */

/* addr.sun_path[0] đã được đặt thành 0 bởi memset() */
strncpy(&addr.sun_path[1], "xyz", sizeof(addr.sun_path) - 2);
/* Abstract name là "xyz" theo sau là các null bytes */

sockfd = socket(AF_UNIX, SOCK_STREAM, 0);
if (sockfd == -1)
    errExit("socket");

if (bind(sockfd, (struct sockaddr *) &addr,
          sizeof(struct sockaddr_un)) == -1)
    errExit("bind");
```

Việc dùng null byte đầu tiên để phân biệt abstract socket name với conventional socket name có thể dẫn đến một hậu quả bất ngờ. Giả sử biến `name` tình cờ trỏ đến chuỗi zero-length và chúng ta cố bind UNIX domain socket vào `sun_path` được khởi tạo như sau: `strncpy(addr.sun_path, name, sizeof(addr.sun_path) - 1)`; Trên Linux, chúng ta sẽ vô tình tạo ra một abstract socket binding — đây rất có thể là lỗi không mong muốn. Trên các triển khai UNIX khác, lời gọi `bind()` tiếp theo sẽ thất bại.

57.7 Tóm tắt (Summary)

UNIX domain sockets cho phép giao tiếp giữa các ứng dụng trên cùng một host. UNIX domain hỗ trợ cả stream sockets và datagram sockets.

Một UNIX domain socket được nhận dạng bởi một pathname trong file system. File permissions có thể được dùng để kiểm soát quyền truy cập vào UNIX domain socket.

System call `socketpair()` tạo một cặp UNIX domain sockets đã được kết nối. Điều này tránh phải thực hiện nhiều system call để tạo, bind và connect các sockets. Một socket pair thường được dùng theo cách tương tự như pipe: một process tạo socket pair, sau đó fork để tạo child kế thừa file descriptor. Hai process sau đó có thể giao tiếp qua socket pair.

Abstract socket namespace đặc thù của Linux cho phép bind một UNIX domain socket vào tên không xuất hiện trong file system.

Đọc thêm: Tham khảo các nguồn thông tin được liệt kê trong Mục 59.15.

57.8 Bài tập (Exercises)

57-1. Trong Mục 57.3, chúng ta đã lưu ý rằng UNIX domain datagram sockets là đáng tin cậy. Hãy viết các chương trình để chứng minh rằng nếu sender truyền datagrams đến UNIX domain datagram socket nhanh hơn tốc độ receiver đọc, thì sender cuối cùng sẽ bị block và vẫn bị block cho đến khi receiver đọc một số datagrams đang chờ.

57-2. Viết lại các chương trình trong Listing 57-3 (`us_xfr_sv.c`) và Listing 57-4 (`us_xfr_cl.c`) để sử dụng Linux-specific abstract socket namespace (Mục 57.6).

57-3. Triển khai lại sequence-number server và client trong Mục 44.8 bằng cách dùng UNIX domain stream sockets.

57-4. Giả sử chúng ta tạo hai UNIX domain datagram sockets được bind vào các path `/somepath/a` và `/somepath/b`, và connect socket `/somepath/a` đến `/somepath/b`. Điều gì xảy ra nếu chúng ta tạo socket datagram thứ ba và cố gửi (`sendto()`) một datagram qua socket đó đến `/somepath/a`? Hãy viết chương trình để xác định câu trả lời. Nếu có quyền truy cập vào các hệ thống UNIX khác, hãy kiểm tra chương trình trên các hệ thống đó để xem câu trả lời có khác không.